Oracle C2M DevOps Release Management & CI/CD Automation

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary — Oracle C2M DevOps Release Management & CI/CD Automation 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 5

Outcome . . . 5

1. Solution Overview — Oracle C2M DevOps Release Management & CI/CD Automation 5

What this solution is . . . 5

The C2M DevOps model — three parts . . . 6

The Dev → QA → Prod pipeline with gates . . . 7

What this is not . . . 7

The deployment model . . . 8

2. Requirements Guide — Oracle C2M DevOps Release Management & CI/CD Automation 8

2.1 A source-control repository for code, config bundles, and settings . . . 8

2.2 Defined pipeline stages, artifacts, and gates . . . 8

2.3 Environment access to run OUAF utilities non-interactively . . . 8

2.4 A promotable code build and configuration bundle . . . 9

2.5 Inbound Web Service deployment access on each target . . . 9

2.6 Routine-admin automation targets (lifecycle, batch, health) . . . 9

2.7 Change control, backups, and a rollback point per environment . . . 9

2.8 A pilot release scope . . . 10

Checklist . . . 10

Global Requirements — Oracle Utilities C2M / WebLogic Solutions 10

1. Licensed Oracle Utilities C2M + WebLogic entitlement . . . 11

2. Certified platform stack (OS, Java, Database, WebLogic) . . . 11

3. Environment topology (Dev / Test / Prod) & access . . . 11

4. Administrative access & tooling (WLST, shell, SQL) . . . 11

5. Change control, backup & a non-production pilot . . . 12

6. Collaborating teams & support model . . . 12

3. Implementation Guide — Oracle C2M DevOps Release Management & CI/CD Automation 12

Step 1 — Assess the environments and the release scope . . . 13

Step 2 — Configure the pipeline stages and routine-admin automation . . . 13

Step 3 — Build stage: produce the release artifacts . . . 13

Step 4 — Migrate-config stage: import configuration into the target . . . 13

Step 5 — Deploy stage: install code, run routine-admin, deploy IWS . . . 14

Step 6 — Validate stage: prove parity and health . . . 14

Step 7 — Validate against the environment gate . . . 14
Step 8 — Pilot Dev → QA, then expand QA → Prod . . . 15
5. Testing Guide — Oracle C2M DevOps Release Management & CI/CD Automation 15
Test 1 — Environments are reachable and at compatible levels . . . 15
Test 2 — Build stage produced the release artifacts . . . 15
Test 3 — Migrate-config stage applied . . . 15
Test 4 — Deploy stage: code build / patch level is identical . . . 16
Test 5 — Inbound Web Services are deployed and active . . . 16
Test 6 — Routine-admin automation left the environment healthy . . . 16
Test 7 — Configuration objects match (no drift) . . . 16
Test 8 — Rollback point is verified (Production) . . . 16
Pass/fail gate . . . 16
6. Troubleshooting Guide — Oracle C2M DevOps Release Management & CI/CD Automation 17
A. A pipeline stage runs by hand but not from the pipeline . . . 17
B. Config is live but behaves wrong — code is out of step . . . 17
C. Inbound Web Services are stale or not active . . . 17
D. The deploy left the environment unhealthy . . . 17
E. The config import review showed unexpected changes . . . 18
F. Compare report shows a gap after a "successful" pipeline run . . . 18
G. A release needs to be rolled back . . . 18
H. Diagnostics quick reference . . . 18
I. When to escalate . . . 19
7. Best Practices — Oracle C2M DevOps Release Management & CI/CD Automation 19
8. Deliverables Summary — Oracle C2M DevOps Release Management & CI/CD Automation 20
Documentation . . . 20
Pipeline stages and artifacts . . . 21
Automation scripts (shell / WLST / SQL — not PowerShell) . . . 21
What the buyer receives . . . 21
Outcome . . . 22
Appendix A — PowerShell Scripts 23

Executive Summary — Oracle C2M DevOps Release Management & CI/CD Automation

Oracle Utilities **Customer to Meter (C2M)** releases are, too often, manual: configuration hand-carried between environments, code that lags the config, Inbound Web Services deployed inconsistently, and a restart-and-batch dance done from memory. The result is slow, error-prone releases where what goes live is not quite what was tested — and the first sign of trouble is a production incident. This solution establishes **DevOps release management and CI/CD** for C2M: a pipeline that **builds, promotes, and deploys** OUAF configuration and code across environments, plus **automation of the routine admin** around each release — so releases are repeatable, governed, and fast, pilot-first and rollback-ready.

This is a complete, do-it-today package written for an experienced C2M / DevOps administrator. A release is defined once as a versioned tuple — build + configuration bundle + IWS list + settings — and run the same way every time, so you build the pipeline once and repeat it across environments and releases instead of improvising each one.

What you get

• A plain-language Solution Overview of the C2M DevOps model: source control for OUAF config and code, a

four-stage **build** → **migrate config** → **deploy** → **validate** pipeline with explicit go/no-go gates, and automation of routine admin (server lifecycle, batch/threadpool quiesce-and-resume, health checks).

• A Requirements Guide with verify / implement-if-missing for every prerequisite — a source-control repo,

defined stages and gates, non-interactive OUAF-utility access, a promotable build and reviewed CMA bundle, `iwsdeploy` access, the routine-admin targets, and change control with a rollback point.

• An Implementation Guide of precise, ordered steps: define the pipeline and its routine-admin automation,

build the artifacts, migrate configuration, deploy code and IWS, and validate parity and health — Dev → QA first, then QA → Prod inside change control.

• Automation scripts (**shell / WLST / SQL**, not PowerShell) for read-only environment/pipeline discovery,

parity/health validation, IWS runtime-state validation via WLST, a source-vs-target compare report, dry-run-by-default remediation, and a release-scope helper.

• Testing and Troubleshooting guides with per-environment pass/fail gates, plus Best Practices for governed,

automated releases.

What this delivers

A repeatable, gated release pipeline — reusable artifacts under source control, code and config in lockstep, routine admin automated, and a validate stage that proves parity and health — so C2M releases go out cleanly with no drift and a verified rollback at every hop.

> ***Boundary.*** *Authored from Oracle's official OUAF / C2M Server Administration documentation. Version-specific utility names, switches, and object names are cited or flagged "verify for your version" (C2M 25.4 / min v2.9, WebLogic 14c); MOS-gated reference detail and patches are pointed to, not reproduced. The single-change CMA/*`iwsdeploy` *promotion mechanics are covered by the companion* ***C2M Environment Migration & Code Promotion*** *solution and referenced here, not duplicated. All pipeline runs assume a non-production environment first and change control for Production.*

The deployment model

Assess, then Configure, then Pilot, then Validate, then Expand — prove the pipeline Dev → QA on a bounded release, pass the gate, then repeat QA → Prod with backups and a verified rollback.

Outcome

C2M releases that are repeatable, governed, and fast — a CI/CD pipeline plus routine-task automation — instead of slow, error-prone manual work.

1. Solution Overview — Oracle C2M DevOps Release Management & CI/CD Automation

What this solution is

This is the repeatable **DevOps model** for Oracle Utilities **Customer to Meter (C2M)**: a pipeline that builds, promotes, and deploys OUAF configuration and code across environments, plus automation for the routine administrative tasks a release depends on (environment restarts, health checks, batch and web-service actions). It is not a one-off script and it is not a promotion of a single change — it is a **named, staged pipeline** you define once and run every release, with explicit go/no-go gates between stages and a documented rollback at each hop.

C2M runs on the **Oracle Utilities Application Framework (OUAF)** on **Oracle WebLogic Server 14c**, and OUAF administration is deliberately command-line and file-based (`splenviron`, `initialSetup`, `configureEnv`, `threadpoolworker`, `iwsdeploy`, and the SQL schema) rather than GUI-only. That is exactly what makes it automatable: every step a release performs has a scriptable, version-controllable form. This solution wires those steps into a governed pipeline.

> ***Version boundary.*** *The OUAF command-line utilities, the Configuration Migration Assistant (CMA),* `iwsdeploy`*, and the install/patch tooling are authored from Oracle's official OUAF / C2M product and Server Administration documentation. Version-specific utility names, switches, batch codes, and object names are* ***cited or flagged "verify for your version"*** *(C2M 25.4 / minimum v2.9, WebLogic 14c per the certification matrix). Some reference detail and patches are gated on* ***My Oracle Support (MOS)****; where a step needs a MOS note or patch, this solution points to it rather than reproducing gated content. All pipeline runs assume a* ***non-production environment first*** *and change control for Production.*

> ***Relationship to the migration solution.*** *The mechanics of *moving one change* — CMA Migration Assistant Configuration, Migration Plans, the Migration Request, the binary export bundle,* `iwsdeploy`*, and the source-vs-target parity compare — are defined in the sibling* ***C2M Environment Migration &***

> ***Code Promotion*** *solution.* ***This solution references that method and does not duplicate it.*** *Here the focus is the layer *above* it: the* ***pipeline*** *that orchestrates build → migrate → deploy → validate as governed, repeatable stages, and the* ***automation*** *of the routine admin tasks around each release.*

The C2M DevOps model — three parts

Part 1 — Source control for OUAF config and code

The release artifacts are put under version control so a release is an auditable, reproducible thing, not a memory:

- **Code and extensions** — custom Java/Groovy, framework hooks, and the OUAF **SDK** project artifacts

live in a source-control repository and produce a versioned **build**.

- **Configuration** — OUAF configuration is exported by CMA as a **binary export bundle** (a file), which

is retained in version control alongside the build. The reviewed bundle that passed a lower environment is the exact artifact promoted onward.

- **Environment settings** — the per-environment `ENVIRON.INI` / `configureEnv` values and property files

(e.g. `threadpoolworker.properties`, `submitbatch.properties`) are captured so an environment can be reconstructed and so differences between environments are visible, not accidental.

A **release** is therefore a named tuple — *build version + configuration bundle + IWS list + environment settings* — that moves through the pipeline together.

Part 2 — The build → promote → deploy pipeline

The pipeline has named, ordered **stages**, each producing an **artifact** and each ending in a **gate**:

1. **Build** — compile the code/extension build from source (OUAF SDK / product install tooling), export

the configuration bundle from the source environment via CMA, and stamp both with the release version. *Artifact:* the versioned build + the CMA bundle. *Gate:* build succeeds, bundle exports clean.

1. **Migrate config** — import the configuration bundle into the target environment (CMA import, reviewed

before apply). *Artifact:* imported configuration. *Gate:* import applied, review showed only the intended scope. (Mechanics per the migration sibling.)

1. **Deploy** — install the same versioned build, run the framework post-install (`initialSetup` and the

build's post-install steps), and deploy the in-scope **Inbound Web Services** with `iwsdeploy` (the online IWS deployment portal is dev-only; QA/Prod are command-line). *Artifact:* the running, deployed target. *Gate:* build level matches, IWS deployment status current.

1. **Validate** — run the read-only parity and health checks (config objects present and matching, IWS

current, build level identical, environment healthy) and prove the target equals the source for the promoted scope. *Artifact:* the validation report. *Gate:* full parity + healthy environment.

The stages run **Dev** → **QA** → **Prod**, one direction only. The *same reviewed artifacts* that passed the QA gate are the ones promoted to Production inside a change window with backups taken first.

Part 3 — Automation of routine admin

Around every release sit routine administrative tasks that are error-prone when done by hand. The pipeline automates them as reusable, named actions so a release is hands-off and consistent:

• **Environment lifecycle** — graceful stop/start of the C2M managed server and admin server, sourced through `splenviron`, so a deploy that needs a restart does not rely on a human remembering the order.

• **Batch / threadpool control** — quiescing and resuming the `threadpoolworkers` (and any online batch submission) around a config/code change, using the OUAF batch utilities, so no batch runs against a half-promoted environment.

• **Health and readiness checks** — post-deploy checks that the domain is up, the IWS are active, the schema is reachable, and the log is clean, wired as an automated gate rather than a manual glance.

• **Promotion glue** — the scripted orchestration that runs the CMA import, the build install, and `iwsdeploy` in the correct order with the correct environment sourced — the repeatable "run the release" action.

The five included scripts are the read-only and dry-run backbone of this automation: discover the pipeline/environment inventory, validate a stage's parity/health, report on it, remediate a gap (dry-run by default), and resolve the release scope.

The Dev → QA → Prod pipeline with gates

Promotion is always one direction and always gated:

• **Dev → QA.** Build from source, export the config bundle, and run the pipeline into QA: migrate config, deploy build + IWS, validate. **Gate:** QA parity confirmed, environment healthy, and QA functional testing passed before anything targets Production.

• **QA → Prod.** Promote the *same* build, the *same* reviewed bundle, and the *same* IWS list to Production inside a change window, with backups taken first and the routine-admin automation (quiesce batch, restart in order) run as part of the deploy. **Gate:** Production parity confirmed against QA and a verified rollback point.

Every gate has an explicit pass/fail check (see the Testing Guide) and a documented rollback (restore the pre-deploy backup, re-import the prior bundle, re-install the prior build).

What this is not

This solution defines and automates the *release pipeline and routine admin*; it does not author your C2M configuration or write your extension code, and it does not re-explain CMA/`iwsdeploy` mechanics (that is the migration sibling). The included scripts are **read-only where they touch live systems** — discovery, validation, and reporting — plus one remediation helper that is **dry-run by default**. They inventory, prove readiness, and preview corrective actions; they do not push config, move code, or restart anything on their own unless explicitly told to.

The deployment model

Assess the environments and the release scope, configure the pipeline and its routine-admin automation, pilot the pipeline Dev → QA on a bounded scope, validate parity and health against the gate, then expand to QA → Prod. The repeatable way: ***Assess, then Configure, then Pilot, then Validate, then Expand.***

2. Requirements Guide — Oracle C2M DevOps Release Management & CI/CD Automation

Verify every requirement before wiring the pipeline. Each entry: Description, Why, Verify, Implement-if-missing. Category-wide prerequisites (entitlement, certified stack, WLST/shell/SQL tooling, change control) are in `global-requirements-oracle.md` and are assumed here. The single-change promotion mechanics (CMA objects, `iwsdeploy`) are covered by the **C2M Environment Migration & Code Promotion** sibling and referenced, not repeated.

2.1 A source-control repository for code, config bundles, and settings

Description: A **version-control repository** (Git or equivalent) that holds the code/extension build sources, the retained CMA **configuration export bundles**, and the per-environment settings snapshots so each release is a reproducible, auditable artifact set. Why: DevOps for C2M starts with treating a release as a versioned tuple (build + bundle + IWS list + settings). Without source control there is no reliable "the same reviewed artifact goes to Prod" — the core promise of the pipeline. Verify: Confirm a repository exists and that a build, a CMA export bundle, and an environment settings snapshot can be checked in and retrieved. `OC2MDEVOPS-Discovery.sh` reports whether the release-artifact locations are present. Implement-if-missing: Stand up a repository and define where builds, bundles, and settings snapshots are retained before running the pipeline.

2.2 Defined pipeline stages, artifacts, and gates

Description: The pipeline's named **stages** (build → migrate config → deploy → validate), the **artifact** each produces, and the **gate** each must pass — written down, not improvised per release. Why: A pipeline is only repeatable if every run performs the same ordered stages with the same pass/fail gate; ad-hoc "do the steps I remember" releases are neither auditable nor safe. Verify: The stage/gate definition exists and names the artifact and gate for each stage; `OC2MDEVOPS-ScopeHelper.sh` prints the release scope (build version, bundle, IWS list) so you confirm a run's inputs match the change. Implement-if-missing: Document the four stages, their artifacts, and their gates (see the Implementation Guide) and register the release scope before the first pilot run.

2.3 Environment access to run OUAF utilities non-interactively

Description: The product OS account and access to source `splenviron` and run the OUAF command-line utilities (`initialSetup`, `configureEnv`, `threadpoolworker`, `iwsdeploy`, the batch/submit utilities) and WLST against each environment the pipeline targets, in a **non-interactive** (scriptable) way. Why: The pipeline automates the release by invoking these utilities; if they can only be run by hand at an interactive prompt, the pipeline cannot orchestrate them. Verify: You can source `splenviron` for the environment and run the utilities from a script/service account; WLST connects to the domain. `OC2MDEVOPS-Discovery.sh` reports environment reachability and the utilities present.

Implement-if-missing: Provision the product/service account, confirm the utilities are on the path under `splenviron`, and (per the global rules) run app-side steps under `sudo -u <product-user>` / the correct JDK, never a bare shell.

2.4 A promotable code build and configuration bundle

Description: The versioned **code/extension build** (produced from the OUAF SDK / product install tooling) and the reviewed **CMA configuration export bundle** for the release — the two artifacts the pipeline promotes together, in lockstep, across environments. Why: Configuration migrates by bundle and code migrates by installed build; a release that promotes one without the other is a half-promotion and a classic source of "config is live but behaves wrong." Verify: A build artifact is identified and versioned, and a CMA export bundle exists for the scope; `OC2MDEVOPS-Validation.py` reports the build level and the presence of the bundle. (Bundle/CMA mechanics: migration sibling.) Implement-if-missing: Produce a versioned build and export the reviewed CMA bundle; retain both in the source-control repository (2.1) as one release.

2.5 Inbound Web Service deployment access on each target

Description: Command-line access to run `iwsdeploy[.sh]` under the product environment (`splenviron`) on every target environment, to deploy the in-scope **Inbound Web Services** as part of the deploy stage. Why: IWS metadata travels with the configuration bundle, but the running service must be deployed — and for QA/Production that deployment is command-line only (the online IWS deployment portal is supported in non-production environments only). Verify: You can source the product environment and run `iwsdeploy` on each target; the *Deployment Status* zone reflects the result. `OC2MDEVOPS-Validation.py` reports each in-scope IWS module's runtime state. Implement-if-missing: Confirm shell access and the in-scope IWS list on each target, and script the `iwsdeploy` invocation into the deploy stage.

2.6 Routine-admin automation targets (lifecycle, batch, health)

Description: The **routine tasks** the pipeline will automate around a release: graceful stop/start of the C2M managed/admin servers, quiescing and resuming the `threadpoolworkers` (and online batch submission), and the post-deploy health/readiness checks (domain up, IWS active, schema reachable, log clean). Why: A release usually needs a controlled restart and a batch quiesce; done by hand these are the most common source of a botched deploy (batch running against a half-promoted schema, servers started out of order). Automating them makes the release consistent and hands-off. Verify: The start/stop order, the batch-quiesce approach, and the health checks are defined for each environment; `OC2MDEVOPS-Validation.py` reports the health signals it can read. Implement-if-missing: Document the lifecycle order and batch-quiesce steps and wire them into the deploy stage; use the included validation as the automated post-deploy health gate.

2.7 Change control, backups, and a rollback point per environment

Description: A change window and approval for each target, a verified **backup** taken before the deploy stage (database/config export, current bundle level, current build/patch level), and a defined rollback. Why: The deploy stage installs a build and imports a bundle that **replaces** existing target configuration; a bad release must be recoverable. Production runs inside change control with a proven restore path. Verify: A backup/restore path is confirmed and a change window/approval exists; the current target bundle and build

levels are recorded so you can re-promote the prior release if needed. Implement-if-missing: Establish the backup/restore procedure and change-control approval before running the pipeline into QA or Prod (see `global-requirements-oracle.md` §5).

2.8 A pilot release scope

Description: A bounded, well-understood first **release** — a single small build + one scoped configuration bundle + a short IWS list — to prove the whole pipeline Dev → QA before running a full release or touching Production. Why: The pipeline, its gates, and its routine-admin automation are validated on a pilot release first, so the stages, artifacts, restart order, and health checks are confirmed end to end cheaply. Verify: A pilot release scope and its owners are identified; `OC2MDEVOPS-ScopeHelper.sh` resolves and prints it. Implement-if-missing: Choose a small, representative release (one bounded change) as the first pipeline run.

Checklist

- Source-control repo for build + config bundle + settings — `OC2MDEVOPS-Discovery.sh`
- Pipeline stages / artifacts / gates defined — `OC2MDEVOPS-ScopeHelper.sh`
- Non-interactive OUAF-utility access per environment — `OC2MDEVOPS-Discovery.sh`
- Promotable code build + reviewed CMA bundle — `OC2MDEVOPS-Validation.py`
- `iwsdeploy` access on each target — `OC2MDEVOPS-Validation.py`
- Routine-admin automation targets defined (lifecycle, batch, health) — `OC2MDEVOPS-Validation.py`
- Backups, change control, and rollback point per environment
- Pilot release scope identified — `OC2MDEVOPS-ScopeHelper.sh`

Global Requirements — Oracle Utilities C2M / WebLogic Solutions

These requirements apply to **every** solution in the **Oracle** category (Oracle Utilities Customer to Meter and its Oracle WebLogic Server middleware). Verify each before starting a solution's steps.

> ***Vendor & version boundary (applies to every Oracle solution).*** *Oracle Utilities* ***Customer to Meter (C2M)****,* ***Customer Care & Billing (CC&B)****,* ***Meter Data Management (MDM)****, and* ***Oracle WebLogic Server*** *are authored from Oracle's official product documentation and the Oracle Utilities Application Framework (OUAF) install/admin guides.* ***Version-specific details*** *(C2M 25.4 / minimum v2.9, WebLogic 14c) are* ***cited or flagged "verify for your version"*** *— never invented. Some Oracle documentation and patches are entitlement-gated on* ***My Oracle Support (MOS)****; where a step requires a MOS note, patch number, or download, the solution points to it rather than reproducing gated content. All work assumes a* ***non-production environment first*** *and change control for production.*

1. Licensed Oracle Utilities C2M + WebLogic entitlement

• **Description:** A licensed Oracle Utilities **Customer to Meter** (and the underlying OUAF) plus the supported **Oracle WebLogic Server 14c** and database (Oracle Database) at the versions in scope.

• **Why required:** Installation media, patches, and support all flow from the entitlement.

• **How to verify:** Confirm the license and access to **My Oracle Support**; check the certified version matrix for C2M ↔ WebLogic ↔ Database ↔ Java.

• **How to implement if missing:** Obtain licensing/entitlement and MOS access before proceeding.

2. Certified platform stack (OS, Java, Database, WebLogic)

• **Description:** A host OS, JDK, Oracle Database, and WebLogic version combination **certified** for the target C2M release (per Oracle's certification/compatibility matrix).

• **Why required:** C2M/OUAF only supports certified combinations; uncertified stacks cause install and patch failures and lose supportability.

• **How to verify:** Compare the environment against the C2M release's certification matrix (MOS).

• **How to implement if missing:** Align OS/JDK/DB/WebLogic to a certified combination first.

3. Environment topology (Dev / Test / Prod) & access

• **Description:** Defined **Development, Test, and Production** environments (and any QA/UAT), with OS accounts (e.g. the OUAF/`splunk`-style product user), WebLogic admin credentials, and DB schema access appropriate to the task.

• **Why required:** C2M is deployed and promoted across environments; each solution assumes the target environment exists and you can reach WebLogic, the file system, and the database.

• **How to verify:** Confirm you can log in to the OS product account, the WebLogic Administration Console / WLST, and the C2M database schema.

• **How to implement if missing:** Stand up / obtain access to the environment before starting.

4. Administrative access & tooling (WLST, shell, SQL)

• **Description:** Administrative access and the standard toolchain: **WLST (Jython)** for WebLogic, the OS **shell** (bash/ksh) for OUAF utilities (e.g. `splmanager`, `configureEnv`, `initialSetup`, batch/threadpool scripts), and **SQL*Plus / SQLcl** for the database.

• **Why required:** C2M/WebLogic administration is performed through these interfaces, not a GUI-only workflow; the solutions' automation is authored as WLST/shell/SQL.

• **How to verify:** Confirm WLST connects to the domain, shell access to the product home, and SQL connectivity to the schema.

• **How to implement if missing:** Provision the access and install the client tooling.

5. Change control, backup & a non-production pilot

• **Description:** A **change-control** process for production changes, verified **backups** (domain config, OUAF product/config, database) before disruptive actions (install, patch, upgrade, migration, tuning), and a **non-production environment** to validate on first.

• **Why required:** Every solution follows **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** with a documented rollback; C2M patches/upgrades and WebLogic changes are high-impact.

• **How to verify:** Confirm a backup/restore path and a change window/approval for production.

• **How to implement if missing:** Establish backups and change control before touching production.

6. Collaborating teams & support model

• **Description:** Engagement with the **infrastructure, DBA, network, application-development, and release/test **teams, and (post go-live) the** hypercare / on-call** support model and monitoring stack (e.g. Dynatrace) the environment uses.

• **Why required:** C2M administration is cross-functional; issues span middleware, database, network, and application layers, and production support runs on a defined escalation and monitoring model.

• **How to verify:** Confirm the named contacts, escalation path, and monitoring tooling.

• **How to implement if missing:** Identify the teams and the support/monitoring model before go-live.

*The deployment model across every Oracle solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes. Automation is delivered as **WLST (Jython), shell, and SQL** scripts — not PowerShell.*

3. Implementation Guide — Oracle C2M DevOps Release Management & CI/CD Automation

Follow these steps in order. This guide **defines the C2M release pipeline** and wires the routine-admin automation: build the release artifacts, migrate configuration, deploy code and Inbound Web Services, validate parity and health — Dev → QA first, then QA → Prod, each behind an explicit gate. Stage names and artifacts are fixed; define the pipeline once and run it every release. The single-change CMA/`iwsdeploy` mechanics are covered in the **C2M Environment Migration & Code Promotion** sibling and are referenced, not repeated, here.

> *Model: prove the pipeline on a bounded release Dev → QA, validate parity + health against the gate, then repeat QA → Prod inside change control. The repeatable way: **Assess, then Configure, then Pilot, then Validate, then Expand.***
>
> ***Direction and safety.** Promotion is always source → target and one-way. Always take a backup of the target before the deploy stage (the pipeline installs a build and imports a bundle that replaces target config). Production is always touched inside a change window with a verified rollback point, with the routine-admin automation (quiesce batch, restart in order) run as part of the deploy.*

Step 1 — Assess the environments and the release scope

1. Run `OC2MDEVOPS-Discovery.sh` against the **source** (Dev) and each **target** (QA, Prod) to inventory

environment reachability, the OUAF release/patch level, the utilities present (`splenviron`, `iwsdeploy`, batch/threadpool), and whether the release-artifact locations (build, bundle, settings) are in place.

1. Confirm source and target are at compatible framework levels; record any gap and do not proceed past a

version mismatch you have not planned for.

1. Define the **release scope** in writing: the code build version, the configuration bundle, the in-scope

Inbound Web Services, and the environment settings that change. Register it so a run's inputs are explicit; confirm with `OC2MDEVOPS-ScopeHelper.sh`.

Step 2 — Configure the pipeline stages and routine-admin automation

1. Define the four **stages** and their **gates** (Requirement 2.2): **Build → Migrate config → Deploy →

Validate**. For each, name the artifact it produces and the pass/fail gate it must clear.

1. Wire the **source-control** locations (Requirement 2.1): where the code build, the CMA export bundle,

and the per-environment settings snapshots are retained, so the same reviewed artifacts flow onward.

1. Wire the **routine-admin automation** (Requirement 2.6) into the deploy stage: the graceful

stop/start order for the C2M managed and admin servers (sourced through `splenviron`), the `threadpoolworker` / batch quiesce-and-resume, and the post-deploy health checks. Use `OC2MDEVOPS-Validation.py` as the automated health gate.

Step 3 — Build stage: produce the release artifacts

1. Compile the versioned **code/extension build** from source (OUAF SDK / product install tooling) and

stamp it with the release version.

1. Export the reviewed **configuration bundle** from the source (Dev) via CMA (mechanics: migration

sibling) and retain it in source control alongside the build.

1. Record the **release tuple** — build version + bundle + IWS list + settings — as one unit. **Gate:** the

build compiles clean and the bundle exports without error.

Step 4 — Migrate-config stage: import configuration into the target

1. Back up the target first (Requirement 2.7): database/config export, and note the current bundle and

build levels so you can re-promote the prior release if needed.

1. Import the configuration bundle into the target (CMA import), **reviewing the proposed changes before

applying** — matching objects are fully replaced from the source; target-only objects are not deleted.

1. **Gate:** the import applied and the review showed only the intended scope. Anything unexpected is your

signal to stop and reconcile (see the migration sibling for the review discipline).

Step 5 — Deploy stage: install code, run routine-admin, deploy IWS

1. **Quiesce** the target with the routine-admin automation: pause the `threadpoolworkers` / online batch

submission so nothing runs against a half-promoted environment, then gracefully stop the C2M managed server if the build requires it.

1. Install the same versioned **code build / patch** that was tested in the source, then run the framework

post-install steps (`initialSetup` and the build's post-install; verify for your version) so the target's file-system artifacts and framework hooks match the source.

1. Deploy the in-scope **Inbound Web Services** with `iwsdeploy[.sh]` under `splenviron` — command-line

for QA and Production (the online IWS deployment portal is non-production only).

1. **Resume** with the routine-admin automation: start the admin/managed servers in order and re-enable the

`threadpoolworkers` / batch. **Gate:** servers up, build level matched, IWS deployment status current.

Step 6 — Validate stage: prove parity and health

1. Run `OC2MDEVOPS-Report.sql` (or the SQL report) to produce the source-vs-target **compare** for the

promoted scope: configuration objects present and matching, code build/patch level identical, IWS status current.

1. Run `OC2MDEVOPS-Validation.py` on the target (WLST) to confirm the server-running and IWS runtime-state

signals with no gaps — this is the environment/health side of the parity gate.

1. If a signal is a gap, remediate the *cause* — re-run the relevant stage (re-import the bundle, re-install

the build, re-deploy an IWS, or re-run a routine-admin step) — then re-validate. Use `OC2MDEVOPS-Remediation.sh` (dry-run by default) to preview any corrective action before applying it.

Step 7 — Validate against the environment gate

1. Complete the Testing Guide for this target. The release passes the gate only when the compare shows

parity for the full scope, the code build level matches the source, IWS deployment status is current, and the environment health checks pass.

1. For **QA**, also confirm functional testing passed on the promoted change before promoting onward.

2. For **Production**, confirm the backup/rollback point is verified and the change window/approval is in place before and after the deploy.

Step 8 — Pilot Dev → QA, then expand QA → Prod

1. **Pilot:** run Steps 3–7 for the bounded pilot release Dev → QA. Passing the QA gate proves the pipeline stages, the routine-admin automation, the IWS deployment, and the parity/health checks all work end to end.

1. **Expand:** run the full release Dev → QA, pass the gate, then promote the **same build, same reviewed bundle, and same IWS list** QA → Prod inside change control, with the routine-admin automation run as part of the deploy. Re-run `OC2MDEVOPS-Report.sql` and the validations for Production and pass the gate.

1. Keep the pipeline definition, the source-control-of-artifacts practice, and the validate stage for every future release — the release is now repeatable, governed, and hands-off.

Order of operations

```
Assess -> Configure pipeline + routine-admin -> Build artifacts -> Migrate config -> Deploy (quiesce, install, iwsdeploy, resume) -> Validate parity/health -> Pass gate -> Pilot Dev→QA -> Expand QA→Prod
```

5. Testing Guide — Oracle C2M DevOps Release Management & CI/CD Automation

Run after each pipeline run (Implementation Steps 4–7), for QA first and then again for Production. Each test confirms one stage of the pipeline landed and that source and target are at parity and healthy. The scripts that touch live systems are read-only; the remediation helper is dry-run by default.

Test 1 — Environments are reachable and at compatible levels

Do: Run `OC2MDEVOPS-Discovery.sh` against source and target. Expected: Both report reachable, with the same OUAF release/patch level (or an intended, understood gap) and the pipeline utilities present; no unplanned version mismatch.

Test 2 — Build stage produced the release artifacts

Do: Confirm the versioned build and the CMA configuration bundle for the release exist in source control; run `OC2MDEVOPS-ScopeHelper.sh` to confirm the release tuple. Expected: The build version, bundle, and IWS list are recorded as one release; the scope matches the intended change.

Test 3 — Migrate-config stage applied

Do: On the target, review the CMA import result for the release, then run `OC2MDEVOPS-Validation.py`. Expected: The import shows applied with no errors and the review showed only the intended scope; validation reports the config-applied signal present for the promoted scope.

Test 4 — Deploy stage: code build / patch level is identical

Do: Run `OC2MDEVOPS-Discovery.sh` on source and target and compare the build/patch level. Expected: The target's code build/patch level equals the source's; configuration and code are in lockstep.

Test 5 — Inbound Web Services are deployed and active

Do: On the target, check the *Deployment Status* zone for the in-scope services and run `OC2MDEVOPS-Validation.py` (WLST). Expected: Each in-scope IWS shows current / active (service revision matches the deployed value, module state active); on QA/Prod deployment was done with `iwsdeploy`, not the portal.

Test 6 — Routine-admin automation left the environment healthy

Do: After the deploy stage's quiesce/install/resume, run `OC2MDEVOPS-Validation.py`. Expected: The admin/managed servers are up in the right order, the `threadpoolworkers` / batch are re-enabled, the schema is reachable, and the health checks pass — no service left stopped or batch left paused.

Test 7 — Configuration objects match (no drift)

Do: Run `OC2MDEVOPS-Report.sql` (or the SQL report) to compare the promoted configuration objects between source and target. Expected: Every object in the release scope is present in the target and matches the source; the compare report shows no missing or mismatched objects for the scope.

Test 8 — Rollback point is verified (Production)

Do: For Production, confirm the pre-deploy backup restores and the prior bundle/build levels are recorded. Expected: A verified restore path exists; the release is recoverable inside the change window.

Pass/fail gate

Gate	Requirement
QA gate	Tests 1–7 pass; configuration parity confirmed, code build matched, IWS active, environment healthy — and QA functional testing passed — before promoting to Production
Production gate	Tests 1–7 pass for Production against QA, and Test 8 (verified rollback point) is satisfied inside the change window

6. Troubleshooting Guide — Oracle C2M DevOps Release Management & CI/CD Automation

Use `OC2MDEVOPS-Discovery.sh` for environment/pipeline inventory, `OC2MDEVOPS-Validation.py` for the WLST parity/health signals on a target (servers running, Inbound Web Service modules active), `OC2MDEVOPS-Report.sql` for the source-vs-target compare, and `OC2MDEVOPS-Remediation.sh` (dry-run by default) to preview corrective actions before applying. All are read-only where they touch live systems, except Remediation which only acts when explicitly told to apply. Single-change CMA/`iwsdeploy` behavior is detailed in the migration sibling.

A. A pipeline stage runs by hand but not from the pipeline

Symptom	Cause	Remediation
An OUAF utility (`iwsdeploy`, `initialSetup`, `threadpoolworker`) works interactively but the pipeline step fails	The pipeline step did not source `splenviron`, so the product environment/path is not set	Source `splenviron` for the environment at the start of the step; confirm the utilities are on the path with `OC2MDEVOPS-Discovery.sh`
A step acts against the wrong environment	The wrong environment was sourced, or `SPLEBASE`/`SPLENVIRON` points elsewhere	Verify the sourced environment matches the target; re-run Discovery to confirm the release/patch level of the environment you are actually on

B. Config is live but behaves wrong — code is out of step

- Symptom: the promoted configuration is present but logic behaves as it did before the change. Cause: the

build was not promoted in lockstep with the configuration bundle — one stage ran without the other. Run `OC2MDEVOPS-Discovery.sh` on source and target and compare the build/patch level; install the matching build, run the framework post-install, and re-validate. Promote the release tuple, not a piece.

C. Inbound Web Services are stale or not active

- The *Deployment Status* zone / `OC2MDEVOPS-Validation.py` shows a service **not current / not active**:

the IWS metadata migrated with the bundle but the service was not (re)deployed in the deploy stage. On the target, source `splenviron` and run `iwsdeploy[.sh]`, then re-check the status.

- IWS deployment "worked in Dev" but not in QA/Prod: the **online IWS deployment portal is supported only in

non-production**. Use the command-line `iwsdeploy` for QA and Production in the deploy stage.

D. The deploy left the environment unhealthy

Symptom	Cause	Remediation
Managed/admin server down or started out of order after a deploy	The routine-admin lifecycle automation did not run, or ran the stop/start in the wrong order	Re-run the lifecycle step in the defined order; confirm with `OC2MDEVOPS-Validation.py` that servers are up
Batch running against a half-promoted schema, or batch left paused	The `threadpoolworker` / batch quiesce-and-resume was skipped or only half-ran	Ensure the deploy stage quiesces *before* install and resumes *after*; re-run the resume step and validate the batch/threadpool health signal
Health check fails after a "successful" deploy	A post-deploy readiness signal (domain up, schema reachable, log clean) is not met	Read `OC2MDEVOPS-Validation.py`, address the failing signal (start the service, fix connectivity), then re-validate

E. The config import review showed unexpected changes

• CMA fully **replaces** matching objects from the source. If the migrate-config review lists objects you

did not intend to change, **stop before applying** — the release scope is wider than the change. Inspect the release scope with `OC2MDEVOPS-ScopeHelper.sh`, narrow it, rebuild the bundle, and review again. (Import/review discipline: migration sibling.)

F. Compare report shows a gap after a "successful" pipeline run

• "The pipeline ran" is not parity. Run `OC2MDEVOPS-Report.sql` to see exactly which of config-objects /

build-level / IWS-status is out of parity, remediate that specific cause by re-running the relevant stage (re-import the bundle, re-install the build, or re-deploy an IWS), and re-compare. Use `OC2MDEVOPS-Remediation.sh` to preview the corrective action first.

G. A release needs to be rolled back

• Restore the pre-deploy backup of the target, or re-promote the previously retained release (recorded prior

bundle and build levels) and re-deploy the prior IWS list, then re-run the routine-admin resume. This is why every deploy is preceded by a backup and every artifact is kept in source control. For Production, perform the rollback inside the same change window.

H. Diagnostics quick reference

```
./OC2MDEVOPS-Discovery.sh   your-env                       # reachability, release/patch, utilities, artif
wlst.sh OC2MDEVOPS-Validation.py your-target               # WLST parity/health: servers running, IWS modu
sqlplus -s cisadm@source @OC2MDEVOPS-Report.sql your-release > report_source.txt   # then run vs target an
./OC2MDEVOPS-ScopeHelper.sh your-release                   # print a release's scope (build, config reques
./OC2MDEVOPS-Remediation.sh your-target --iws your-iws-app # dry-run by default; add --apply to act
```

I. When to escalate

If source and target are at the same level, every stage ran without error, IWS read active, the build levels match, and the environment health checks pass — but the compare report still shows a gap after remediation — capture the `OC2MDEVOPS-Discovery.sh`, `OC2MDEVOPS-Validation.py`, and `OC2MDEVOPS-Report.sql` output and review the pipeline stage definitions and release scope with the application-development and release teams. For CMA/`iwsdeploy` reference behavior or a suspected framework defect, raise it on My Oracle Support with the captured output (verify the note/patch for your version).

7. Best Practices — Oracle C2M DevOps Release Management & CI/CD Automation

• Treat a release as one versioned tuple. Build version + configuration bundle + IWS list + environment

settings move together, retained in source control; a bundle without its matching build (or a build without its config) is a half-release. Promote the tuple, not a piece of it.

• Put every release artifact under version control. The code build, the binary CMA export bundle, and the

per-environment settings are just files — retain them so the exact reviewed artifacts that passed QA are the ones applied to Production, and so every release is auditable and reproducible.

• Define the stages and gates once, run them the same way every release. Build → migrate config → deploy →

validate, each with a named artifact and an explicit pass/fail gate. "Do the steps I remember" is neither repeatable nor safe.

• Automate the routine admin, don't do it by hand. The graceful stop/start order, the batch/threadpool

quiesce-and-resume, and the post-deploy health checks are the most common source of a botched deploy; script them into the deploy stage so a release is consistent and hands-off.

• Always quiesce batch before you promote. Pause the `threadpoolworkers` / online batch submission so no

job runs against a half-promoted schema, then resume as part of the deploy — never leave batch running through a config/code change.

• Source `splenviron` for every scripted step. OUAF utilities (`initialSetup`, `iwsdeploy`,

`threadpoolworker`, batch) depend on the product environment; a pipeline step that runs without it will fail or, worse, act against the wrong environment.

• Deploy IWS from the command line for QA and Prod. The online IWS deployment portal is supported only in

non-production; use `iwsdeploy` under `splenviron` for controlled environments and confirm the Deployment Status zone / WLST state reads current and active.

• Back up the target before every deploy stage. Because the deploy installs a build and imports a bundle

that replaces objects, the pre-deploy backup (plus the recorded prior bundle/build levels) is your rollback; take it every time, not just for Production.

• Validate parity *and* health, don't assume. "The pipeline ran" is not "the target equals the source and is healthy." Run the compare and the health/IWS validations on every release; parity + healthy confirmed is the evidence of a clean deploy.

• Promote one direction, through the gates. Dev → QA → Prod, never sideways or backward; pass the explicit gate (parity + health + functional test) at each hop before the next, with the *same* artifacts.

• Prove the pipeline on a pilot release first. Run one bounded release Dev → QA before running a full release or touching Production — the pilot gate catches stage, restart-order, IWS, and health-check gaps cheaply.

• Never author configuration or hand-tune settings directly in QA or Prod. All change originates in Dev and flows through the pipeline; a hand-edit downstream is exactly the drift and the un-reproducible release this method exists to prevent.

• Keep source and target at compatible levels. Same-release promotion is the safe default; treat a framework-version gap as its own planned exercise, not an incidental part of a release.

8. Deliverables Summary — Oracle C2M DevOps Release Management & CI/CD Automation

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The C2M DevOps model: source control for OUAF config/code, the build → promote → deploy pipeline, and automation of routine admin (the why and the moving parts)
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Oracle)	Category-wide prerequisites (entitlement, certified stack, WLST/shell/SQL, change control), published once
3. Implementation Guide	Exact, ordered steps to define the pipeline (build → migrate config → deploy → validate), wire routine-admin automation, add gates/rollback, and validate — Dev → QA → Prod
5. Testing Guide	Validation procedures with per-environment pass/fail gates

Document	Purpose
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations for governed, automated releases

Pipeline stages and artifacts

Stage	Artifact	Gate
Build	Versioned code build + reviewed CMA configuration bundle	Build compiles clean; bundle exports without error
Migrate config	Imported configuration in the target (CMA import, reviewed)	Import applied; review showed only the intended scope
Deploy	Running target: installed build + deployed IWS, routine-admin run	Build level matched; IWS current; servers up
Validate	The parity + health report	Full parity for the scope; environment healthy

Automation scripts (shell / WLST / SQL — not PowerShell)

Script	What it does
`OC2MDEVOPS-Discovery.sh`	Read-only inventory of an environment/pipeline: reachability, OUAF release/patch level, utilities present, and release-artifact locations (build, bundle, settings)
`OC2MDEVOPS-Validation.py`	Read-only WLST parity/health check on a target: C2M servers running and in-scope Inbound Web Service modules active (vs stale/not-deployed)
`OC2MDEVOPS-Report.sql`	Read-only source-vs-target compare report for a promoted scope (build level, CMA import history, IWS revisions, batch controls); run against source and target and diff
`OC2MDEVOPS-Remediation.sh`	Corrective helper for parity/health gaps (re-import bundle, re-install build, re-deploy IWS, resume batch); **dry-run by default**, prints intended actions before any apply
`OC2MDEVOPS-ScopeHelper.sh`	Resolves and prints a release's scope (build artifact, config request, IWS list) so you promote exactly what you intend

What the buyer receives

- This implementation guide as a branded Word document and PDF.

- All automation scripts (shell / WLST / SQL) in a single ZIP.
- A repeatable release pipeline following the model: Assess, then Configure, then Pilot, then Validate, then Expand.

Outcome

C2M releases that are repeatable, governed, and fast — a CI/CD pipeline (build → migrate config → deploy → validate) plus automation of the routine admin around each release — so what you tested is exactly what goes live, pilot-first and rollback-ready, instead of slow, error-prone manual work.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

OC2MDEVOPS-Discovery.sh

Purpose: Inventory one Oracle Utilities C2M / OUAF environment so you can assess it before wiring or running the release pipeline: whether the environment is reachable, its OUAF release/patch level, which pipeline utilities are present (splenviron, iwsdeploy, threadpoolworker, batch), and whether the release-artifact locations (code build, CMA config bundle, settings snapshot) are in place. Read-only — it inventories the environment, it never builds, imports, deploys, restarts, or changes anything. Run it against your-source-env and each your-target-env. HOW TO RUN source the product environment first, then: ./OC2MDEVOPS-Discovery.sh your-env-name e.g. ./OC2MDEVOPS-Discovery.sh QA Requires: product OS account with splenviron access and SQL*Plus (or SQLcl) to the C2M schema. Point OC2MDEVOPS_ARTIFACTS at your build/bundle/settings root; set the schema connect string in OC2MDEVOPS_DB below or export OC2MDEVOPS_DB. EXPECTED RESULTS A one-screen summary: reachability, release/patch level, a PASS/GAP line per pipeline utility, a PASS/GAP line for each release-artifact location, and CMA/IWS object counts. Record these against the Requirements checklist. GAP lines are what you remediate before configuring. NOTES White-label, the repeatable way. Read-only against a live system. Object/utility names follow OUAF conventions; verify names for your C2M version (C2M 25.4 / min v2.9, WebLogic 14c). MOS-gated reference detail is pointed to, never reproduced.

How to run: source the product environment first, then: ./OC2MDEVOPS-Discovery.sh your-env-name e.g. ./OC2MDEVOPS-Discovery.sh QA Requires: product OS account with splenviron access and SQL*Plus (or SQLcl) to the C2M schema. Point OC2MDEVOPS_ARTIFACTS at your build/bundle/settings root; set the schema connect string in OC2MDEVOPS_DB below or export OC2MDEVOPS_DB. EXPECTED RESULTS A one-screen summary: reachability, release/patch level, a PASS/GAP line per pipeline utility, a PASS/GAP line for each release-artifact location, and CMA/IWS object counts. Record these against the Requirements checklist. GAP lines are what you remediate before configuring. NOTES White-label, the repeatable way. Read-only against a live system. Object/utility names follow OUAF conventions; verify names for your C2M version (C2M 25.4 / min v2.9, WebLogic 14c). MOS-gated reference detail is pointed to, never reproduced.

OC2MDEVOPS-Remediation.sh

Purpose: Close the common gaps the validate stage turns up on a target: re-import the retained CMA config bundle, re-install/post-install the promoted code build, re-deploy an in-scope Inbound Web Service with iwsdeploy, or re-run a routine-admin action (resume the threadpoolworker / restart a server in order). DRY-RUN by default — it prints the exact command it would run and changes nothing until you pass --apply. Even with --apply it acts only on the target you name and the bundle/build/iws/action you name; it never invents scope. HOW TO RUN dry-run (default, safe) prints the plan: ./OC2MDEVOPS-Remediation.sh your-target-env --bundle your-bundle-file --iws your-iws-app --resume-batch to actually perform it, add --apply (source splenviron first; run in the change window): ./OC2MDEVOPS-Remediation.sh your-target-env --iws your-iws-app --apply Options: --bundle FILE --build DIR --iws APP --resume-batch --apply EXPECTED RESULTS Dry-run: a numbered plan of the exact import / install / iwsdeploy / batch-resume commands, then "DRY-RUN — nothing changed." With --apply: each step runs and reports success/failure; stop on the first failure and re-validate with OC2MDEVOPS-Report.sql before continuing. NOTES White-label, the repeatable way. DRY-RUN by

default; --apply is the only way it mutates, and only inside a change window with a verified backup. QA/Prod IWS deployment is command-line iwsdeploy (the portal is dev-only). Verify utility names/switches for your C2M version (C2M 25.4 / min v2.9).

How to run: dry-run (default, safe) prints the plan: ./OC2MDEVOPS-Remediation.sh your-target-env --bundle your-bundle-file --iws your-iws-app --resume-batch to actually perform it, add --apply (source splenviron first; run in the change window): ./OC2MDEVOPS-Remediation.sh your-target-env --iws your-iws-app --apply Options: --bundle FILE --build DIR --iws APP --resume-batch --apply EXPECTED RESULTS Dry-run: a numbered plan of the exact import / install / iwsdeploy / batch-resume commands, then "DRY-RUN — nothing changed." With --apply: each step runs and reports success/failure; stop on the first failure and re-validate with OC2MDEVOPS-Report.sql before continuing. NOTES White-label, the repeatable way. DRY-RUN by default; --apply is the only way it mutates, and only inside a change window with a verified backup. QA/Prod IWS deployment is command-line iwsdeploy (the portal is dev-only). Verify utility names/switches for your C2M version (C2M 25.4 / min v2.9).

OC2MDEVOPS-Report.sql

Purpose: Produce the validate-stage evidence that a release reached parity. Run once against the source schema and once against each target schema, then diff the two outputs: it lists the code build/patch level, the CMA import history for the release, the in-scope Inbound Web Service revisions, and the batch-control set (routine-admin scope) — each with a stable sort so the two outputs diff cleanly. Matching output across environments = parity for the scope. Read-only — SELECT only, it changes nothing. HOW TO RUN run against source, then against target, capturing each to a file, then compare: sqlplus -s cisadm@source @OC2MDEVOPS-Report.sql your-release-code > report_source.txt sqlplus -s cisadm@target @OC2MDEVOPS-Report.sql your-release-code > report_target.txt diff report_source.txt report_target.txt Pass the release / migration-request code as the first argument (defaults to a placeholder). EXPECTED RESULTS A stable, sorted listing (build level, CMA import history, IWS revisions, batch controls) for the release scope. An empty diff between source and target = PASS (parity, no drift). Any diff names exactly which build/config/IWS element is missing or mismatched on the target. NOTES White-label, the repeatable way. Read-only. Table/column names follow OUAF conventions; verify names for your C2M version (C2M 25.4 / min v2.9). Detailed CMA/iwsdeploy reference behavior is on MOS and in the migration sibling — pointed to, not reproduced here.

How to run: run against source, then against target, capturing each to a file, then compare: sqlplus -s cisadm@source @OC2MDEVOPS-Report.sql your-release-code > report_source.txt sqlplus -s cisadm@target @OC2MDEVOPS-Report.sql your-release-code > report_target.txt diff report_source.txt report_target.txt Pass the release / migration-request code as the first argument (defaults to a placeholder). EXPECTED RESULTS A stable, sorted listing (build level, CMA import history, IWS revisions, batch controls) for the release scope. An empty diff between source and target = PASS (parity, no drift). Any diff names exactly which build/config/IWS element is missing or mismatched on the target. NOTES White-label, the repeatable way. Read-only. Table/column names follow OUAF conventions; verify names for your C2M version (C2M 25.4 / min v2.9). Detailed CMA/iwsdeploy reference behavior is on MOS and in the migration sibling — pointed to, not reproduced here.

OC2MDEVOPS-ScopeHelper.sh

Purpose: Show exactly what a release will promote before you run the pipeline, so you deploy what you intend and nothing more. It resolves the release tuple — the code build version, the CMA config bundle / migration-request scope, and the in-scope Inbound Web Services — and flags any element that is missing

(no build recorded, request not found, an IWS not defined). Read-only — it inspects the release and environment definitions, it never builds, imports, deploys, or restarts. HOW TO RUN source the product environment, then: ./OC2MDEVOPS-ScopeHelper.sh your-release-code e.g. ./OC2MDEVOPS-ScopeHelper.sh your-2026-r1-release Point OC2MDEVOPS_ARTIFACTS at your build/bundle root; set OC2MDEVOPS_IWS to the in-scope IWS list; set the schema connect string in OC2MDEVOPS_DB below or export OC2MDEVOPS_DB. EXPECTED RESULTS A readable listing of the release scope — build artifact present?, config request found?, each in-scope IWS defined? — with a PASS/GAP line per element. PASS across all means the release is ready to run through the pipeline; any GAP names the element to fix before the build stage. NOTES White-label, the repeatable way. Read-only against a live system. A release is one versioned tuple (build + bundle + IWS list + settings) — resolve it once, run it every environment. Table/column names follow OUAF conventions; verify names for your C2M version (C2M 25.4 / min v2.9).

How to run: source the product environment, then: ./OC2MDEVOPS-ScopeHelper.sh your-release-code e.g. ./OC2MDEVOPS-ScopeHelper.sh your-2026-r1-release Point OC2MDEVOPS_ARTIFACTS at your build/bundle root; set OC2MDEVOPS_IWS to the in-scope IWS list; set the schema connect string in OC2MDEVOPS_DB below or export OC2MDEVOPS_DB. EXPECTED RESULTS A readable listing of the release scope — build artifact present?, config request found?, each in-scope IWS defined? — with a PASS/GAP line per element. PASS across all means the release is ready to run through the pipeline; any GAP names the element to fix before the build stage. NOTES White-label, the repeatable way. Read-only against a live system. A release is one versioned tuple (build + bundle + IWS list + settings) — resolve it once, run it every environment. Table/column names follow OUAF conventions; verify names for your C2M version (C2M 25.4 / min v2.9).

OC2MDEVOPS-Validation.py

Purpose: Confirm a C2M release landed cleanly on a target from the WebLogic side after the deploy stage: that the domain is reachable, the C2M managed and admin servers are in the running state, and the in-scope Inbound Web Service (IWS) modules are deployed and active (not stale). Pair it with OC2MDEVOPS-Report.sql (config-object + build-level compare) for the full validate-stage gate. Read-only — it reads deployment and server state through WLST, it never deploys, redeploys, starts, stops, or changes anything. HOW TO RUN source the product environment, then run under the OUAF WLST wrapper, e.g. wlst.sh OC2MDEVOPS-Validation.py your-target-env Connection comes from the environment (ENVIRON.INI / wlst connect), or export: OC2MDEVOPS_ADMIN_URL e.g. t3://your-admin-host:7001 OC2MDEVOPS_WLS_USER WebLogic admin user (read-only monitor role is sufficient) OC2MDEVOPS_IWS comma-separated in-scope IWS app/module names for this release The password is read from OC2MDEVOPS_WLS_PASS if set, otherwise WLST prompts (never hard-coded). EXPECTED RESULTS A domain-reachability line, a PASS/GAP line per server (running vs not), and a PASS/GAP line per in-scope IWS module (active vs not-deployed/stale). All PASS means the WebLogic side of parity and health is met; any GAP names the server to start or the module to (re)deploy with iwsdeploy. Non-zero exit if any GAP is found. NOTES White-label, the repeatable way. Read-only against a live domain. WLST/Jython, not PowerShell. The online IWS deployment portal is dev-only; QA/Prod use command-line iwsdeploy. Verify MBean paths / server / service names for your WebLogic 14c / C2M version (C2M 25.4 / min v2.9).

How to run: source the product environment, then run under the OUAF WLST wrapper, e.g. wlst.sh OC2MDEVOPS-Validation.py your-target-env Connection comes from the environment (ENVIRON.INI / wlst connect), or export: OC2MDEVOPS_ADMIN_URL e.g. t3://your-admin-host:7001 OC2MDEVOPS_WLS_USER WebLogic admin user (read-only monitor role is sufficient)

OC2MDEVOPS_IWS comma-separated in-scope IWS app/module names for this release The password is read from OC2MDEVOPS_WLS_PASS if set, otherwise WLST prompts (never hard-coded). EXPECTED RESULTS A domain-reachability line, a PASS/GAP line per server (running vs not), and a PASS/GAP line per in-scope IWS module (active vs not-deployed/stale). All PASS means the WebLogic side of parity and health is met; any GAP names the server to start or the module to (re)deploy with iwsdeploy. Non-zero exit if any GAP is found. NOTES White-label, the repeatable way. Read-only against a live domain. WLST/Jython, not PowerShell. The online IWS deployment portal is dev-only; QA/Prod use command-line iwsdeploy. Verify MBean paths / server / service names for your WebLogic 14c / C2M version (C2M 25.4 / min v2.9).